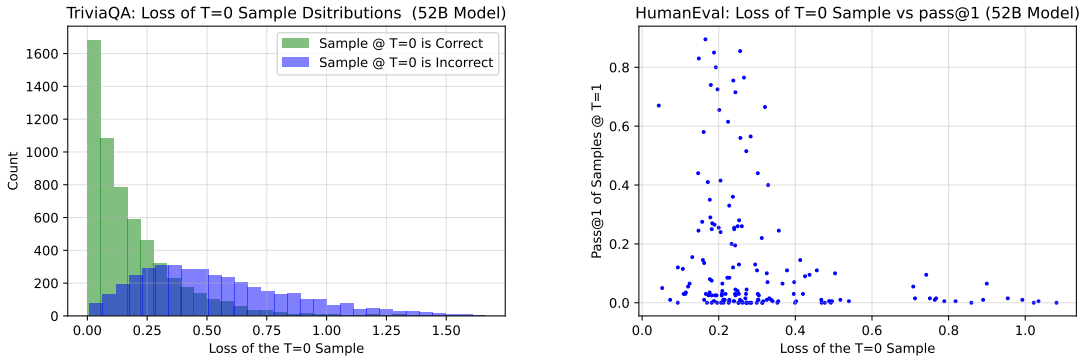


Figure 22 Using the model's loss on the $T = 0$ sample to differentiate between problems it knows the answer to and problems it doesn't know the answer to. **Left**: On TriviaQA, we see that the loss on the $T = 0$ sample is more commonly lower for answers that are correct. **Right**: On Codex HumanEval, we see that the loss on the $T = 0$ sample is somewhat indicative of the accuracy of the T=1 samples. Specifically, we see that our model performs poorly in terms of pass@k on all questions that resulted in a high loss on the T=0 sample.

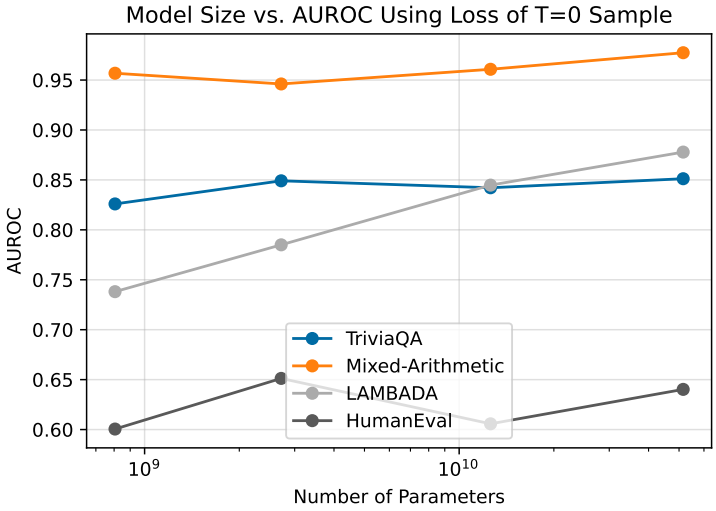


Figure 23 Model size trends of the AUROC, when using the loss of the T = 0 sample to predict whether or not the model will get the question correct. For the purposes of this chart, we consider HumanEval problems with a pass@10 > 0.5 as problems that the model got 'correct'.

B Discriminating What Models Know with Entropy or Loss

In this section we will compare methods that attempt to discriminate correct and incorrect samples, without achieving calibration.

B.1 Loss of the Sample

A very simple method is to generate a sample, and then look at the model's loss on this sample, averaged over all tokens. This has some power to discriminate between correct and incorrect samples, as shown in Figure 22 and 23.

We considered summing the loss across all tokens as well. However, this would make the length of the candidate solution a confounding variable. For many evals, such as Codex HumanEval, more difficult questions naturally require longer answers.

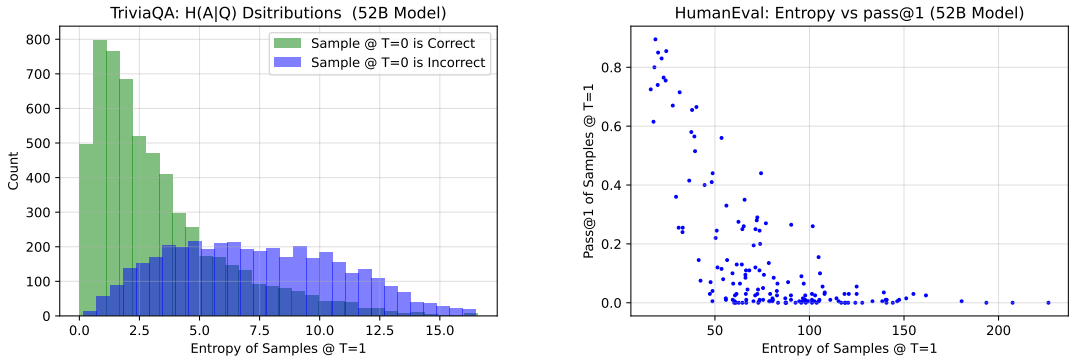


Figure 24 Entropy of samples from the 52B model on TriviaQA and HumanEval. On the left, we see that the TriviaQA questions that the model gets correct and incorrect at $T = 0$ have different entropy distributions. The average entropy of the questions the model gets correct is lower than that of the questions the model gets incorrect. On the right, we compute the entropy and pass@1 over many samples from each question in HumanEval. We see that as the entropy of the answer distribution decreases, models get higher pass@1 scores.

B.2 Entropy of the Answer Distribution

One hypothesis about model hallucinations is that they are diverse. Specifically, we hypothesize that when a model knows the answer to a particular question, it is confident in its response, and sampling the answer many times at $T = 1$ would result in an answer distribution with small entropy. Conversely, when a model is unsure about its response, it will start "hallucinating" responses, leading to an answer distribution with high entropy. In order to investigate this, we sample $N = 200$ answers from each question and estimate the entropy of the answer distribution for a given question $H(A|Q)$ as:

$$H(A|Q) = \mathbb{E}_{A \text{ sampled from model}}\left[-\log \mathbb{P}(A|Q)\right] = \mathbb{E}_A\left[\sum_{a_i \text{ tokens of A}} -\log \mathbb{P}(a_i|Q, a_0, \cdots a_{i-1})\right]$$

In Figure 24, we look at how well $H(A|Q)$ performs as a signal for which questions the model gets correct vs. incorrect. The distribution of $H(A|Q)$ looks different depending on whether or not the model got the question correct, indicating that it has some predictive power. In Figure 25, we show model size trends for the AUROC using $H(A|Q)$ for many evals. It's interesting to note the negative trend of AUROC on HumanEval as we increase model size. We found that larger models solve harder HumanEval problems, but also often give a diverse set of correct answers. This is in conflict with the original hypothesis that when a model knows the answer to a question, its answer distribution is not diverse. Because larger models are able to give more diverse *and correct* answers, we believe that $H(A|Q)$ is not a robust measure of whether or not a model knows the answer to a question, especially as models increase in size.

B.3 Loss of Sampled Answers Stuffed into a Single Context

Another way to measure diversity is to stuff many answer samples in the context of an LLM and compute the standard token-level language modelling loss on the stuffed context. We can then use the loss on the entire stuffed context as a measure of if the model knows the answer or not. The idea here is similar to the idea with the Entropy method - questions that the model knows the answer to will have less diverse answer distributions.

Here, we stuff 30 model-generated samples at $T = 1$ in the context window and compute the mean token loss across the sequence. We choose to take the mean token loss so that results aren't confounded by the length of the sampled answers, since for benchmarks like HumanEval, the length of correct answers tends to be correlated with the difficulty of the question. See Figure 26 and Figure 27 for results. We see that this method has some discriminative power for differentiating whether or not the model knows the answer. However, as expected, we see the a negative model size trend for Codex HumanEval. See Section B.2 for a short discussion on this negative trend - we believe that the same intuition applies here.

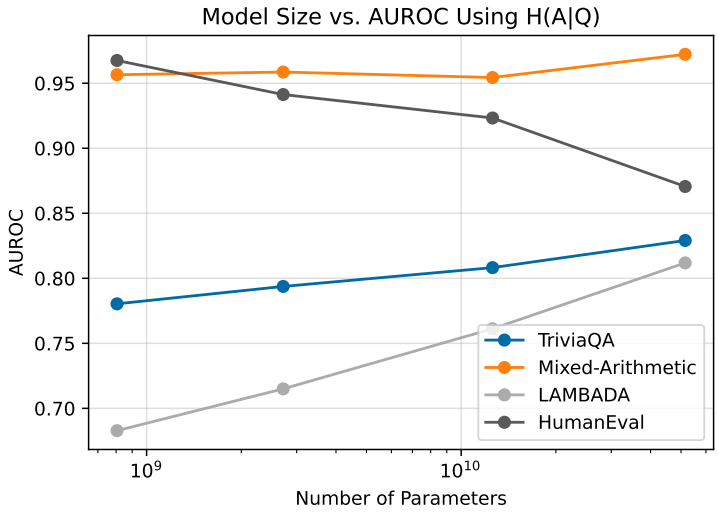


Figure 25 Model size vs. AUROC trends for several evals using the entropy of the answer distribution. Here we are attempting to distinguish whether the model knows the answer to given questions, not whether individual samples are valid. For the Codex HumanEval we threshold "correct" answers as those with Pass@10 > 0.5, whereas we use Pass@1 in the main text.

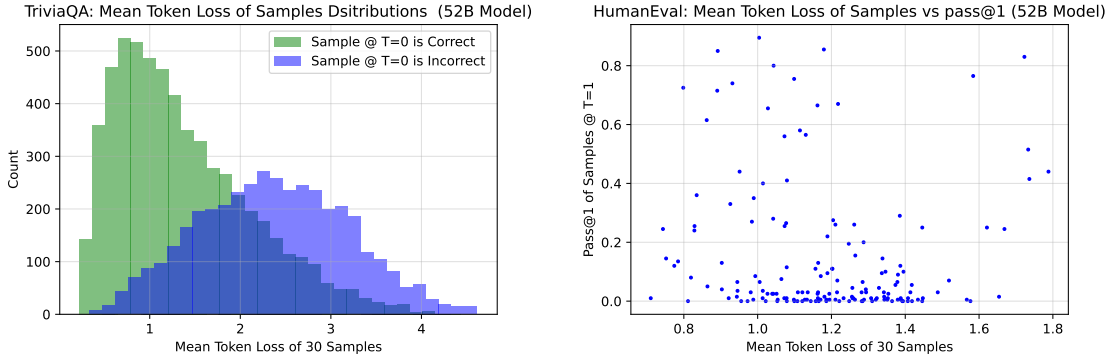


Figure 26 Token Loss of stuffed answers can sometimes provide signal as to whether the model knows the answer to a given question or not. On TriviaQA, there is a clear difference between the token loss of 30 samples when the model gets the underlying question correct vs. incorrect. On Codex HumanEval, there is much less of a correlation between these metrics.

C More P(True) Evaluation Results and Details

Here we include some examples of the behavior of True/False self-evaluation 0-shot. In Figure 38 we see that models do a good job *separating* correct and incorrect samples, but their calibration for P(True) is quite poor. To obtain good calibration one must evaluate few-shot, as seen in Figure 28.

D Mixed-Arithmetic and Function Synthesis Dataset Descriptions

Our Mixed-Arithmetic dataset is a programmatically generated set of arithmetic problems. Table 4 gives an overview of the types of questions that were contained in the dataset.

Our Python function synthesis dataset was created by scraping Python repositories from GitHub that contained at least one file with 'test' in its name. For each repository, we called pytest with coverage, and grabbed all non-test functions where at least half the lines were covered by passing test(s). We then used these tests